

铁道供电作业视频智能评价与实训考核平台源代码

软件名称：铁道供电作业视频智能评价与实训考核平台 V1.0

版本号：V1.0

文档类型：源代码

目录

1. backend/app/database.py
2. backend/app/main.py
3. backend/app/models.py
4. backend/app/routers/__init__.py
5. backend/app/routers/analysis.py
6. backend/app/routers/catalog.py
7. backend/app/routers/submissions.py
8. backend/app/routers/tasks.py
9. backend/app/seed.py
10. backend/app/services/ai/__init__.py
11. backend/app/services/ai/action_recognizer.py
12. backend/app/services/ai/config.py
13. backend/app/services/ai/contracts.py
14. backend/app/services/ai/frame_extractor.py
15. backend/app/services/ai/mock_analyzer.py
16. backend/app/services/ai/multimodal.py
17. backend/app/services/ai/pipeline.py
18. backend/app/services/ai/pose_estimator.py
19. backend/app/services/ai/steps.py
20. backend/app/services/ai/yolo_detector.py
21. backend/app/services/analyzer.py
22. backend/app/services/report.py
23. backend/app/services/rule_engine.py
24. backend/requirements.txt
25. backend/smoke_test.py
26. frontend/index.html
27. frontend/package-lock.json
28. frontend/package.json
29. frontend/src/App.vue
30. frontend/src/main.js
31. frontend/src/services/api.js
32. frontend/src/styles.css
33. frontend/vite.config.js

backend/app/database.py

```
from pathlib import Path

from sqlmodel import SQLModel, Session, create_engine

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
UPLOAD_DIR = DATA_DIR / "uploads"
DATABASE_URL = f"sqlite:///{'DATA_DIR' / 'app.db'}"

DATA_DIR.mkdir(parents=True, exist_ok=True)
UPLOAD_DIR.mkdir(parents=True, exist_ok=True)

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})

def init_db() -> None:
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session
```

backend/app/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from sqlmodel import Session

from .database import engine, init_db
from .routers import analysis, catalog, submissions, tasks
from .seed import seed_defaults

app = FastAPI(title="铁道供电作业视频智能评价平台", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.on_event("startup")
def on_startup() -> None:
    init_db()
    with Session(engine) as session:
        seed_defaults(session)

@app.get("/api/health")
def health():
    return {"status": "ok"}

app.include_router(catalog.router)
app.include_router(tasks.router)
app.include_router(submissions.router)
app.include_router(analysis.router)
```

backend/app/models.py

```
from datetime import datetime
from enum import Enum
from typing import Optional

from sqlmodel import Field, SQLModel


class AnalysisStatus(str, Enum):
    pending = "pending"
    running = "running"
    completed = "completed"
    failed = "failed"


class TrainingTask(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    title: str
    course: str
    class_name: str
    teacher: str
    description: str = ""
    created_at: datetime = Field(default_factory=datetime.utcnow)


class StandardStep(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    code: str = Field(index=True, unique=True)
    name: str
    order_index: int = Field(index=True)
    description: str = ""
    required: bool = True


class ScoringRule(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    code: str = Field(index=True, unique=True)
    title: str
    rule_type: str
    target_code: str = ""
    deduction: float
    severity: str = "medium"
    description: str = ""


class VideoSubmission(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    task_id: int = Field(foreign_key="trainingtask.id", index=True)
    student_name: str
    student_no: str = ""
    original_filename: str
    stored_filename: str
    content_type: str = ""
    size_bytes: int = 0
    uploaded_at: datetime = Field(default_factory=datetime.utcnow)


class AnalysisJob(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    submission_id: int = Field(foreign_key="videosubmission.id", index=True)
    status: AnalysisStatus = Field(default=AnalysisStatus.pending, index=True)
    score: float = 0
    summary: str = ""
    error_message: str = ""
    created_at: datetime = Field(default_factory=datetime.utcnow)
    started_at: Optional[datetime] = None
    completed_at: Optional[datetime] = None
```

```

class DetectionResult(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    job_id: int = Field(foreign_key="analysisjob.id", index=True)
    label: str
    confidence: float
    timestamp_sec: float
    bbox: str = ""

class ActionResult(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    job_id: int = Field(foreign_key="analysisjob.id", index=True)
    action: str
    confidence: float
    start_sec: float
    end_sec: float

class StepEvent(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    job_id: int = Field(foreign_key="analysisjob.id", index=True)
    step_code: str = Field(index=True)
    step_name: str
    start_sec: float
    end_sec: float
    confidence: float
    evidence: str = ""

class Violation(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    job_id: int = Field(foreign_key="analysisjob.id", index=True)
    rule_code: str
    title: str
    deduction: float
    severity: str
    timestamp_sec: Optional[float] = None
    reason: str
    suggestion: str

class EvaluationReport(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    job_id: int = Field(foreign_key="analysisjob.id", index=True, unique=True)
    score: float
    conclusion: str
    strengths: str
    problems: str
    suggestions: str
    generated_at: datetime = Field(default_factory=datetime.utcnow)

class KnowledgeDocument(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    title: str
    category: str = "规程"
    content: str
    created_at: datetime = Field(default_factory=datetime.utcnow)

```

backend/app/routers/__init__.py

backend/app/routers/analysis.py

```

from datetime import datetime

from fastapi import APIRouter, Depends, HTTPException
from sqlmodel import Session, select

```

```

from ..database import UPLOAD_DIR, engine, get_session
from ..models import (
    ActionResult,
    AnalysisJob,
    AnalysisStatus,
    DetectionResult,
    EvaluationReport,
    StepEvent,
    VideoSubmission,
    Violation,
)
from ..services.analyzer import Analyzer
from ..services.report import generate_report
from ..services.rule_engine import evaluate_rules

router = APIRouter(prefix="/api/analysis", tags=["analysis"])

def run_analysis(job_id: int) -> None:
    with Session(engine) as session:
        job = session.get(AnalysisJob, job_id)
        if not job:
            return
        job.status = AnalysisStatus.running
        job.started_at = datetime.utcnow()
        session.add(job)
        session.commit()

        try:
            submission = session.get(VideoSubmission, job.submission_id)
            if not submission:
                raise RuntimeError("Submission not found")
            facts = Analyzer().analyze(UPLOAD_DIR / submission.stored_filename, submission.original_filename)

            for fact in facts.detections:
                session.add(DetectionResult(job_id=job_id, **fact.__dict__))
            for fact in facts.actions:
                session.add(ActionResult(job_id=job_id, **fact.__dict__))
            for fact in facts.steps:
                session.add(StepEvent(job_id=job_id, **fact.__dict__))
            session.commit()

            score, _ = evaluate_rules(session, job_id)
            report = generate_report(session, job_id, score)
            session.add(report)

            job.score = score
            notes = f" 模型备注: {'; '.join(facts.notes)}" if facts.notes else ""
            job.summary = f"{report.conclusion}{notes}"
            job.status = AnalysisStatus.completed
            job.completed_at = datetime.utcnow()
            session.add(job)
            session.commit()
        except Exception as exc:
            job.status = AnalysisStatus.failed
            job.error_message = str(exc)
            job.completed_at = datetime.utcnow()
            session.add(job)
            session.commit()

@router.get("/jobs")
def list_jobs(session: Session = Depends(get_session)):
    return session.exec(select(AnalysisJob).order_by(AnalysisJob.created_at.desc())).all()

@router.get("/jobs/{job_id}")
def get_job(job_id: int, session: Session = Depends(get_session)):
    job = session.get(AnalysisJob, job_id)
    if not job:

```

```

        raise HTTPException(status_code=404, detail="Job not found")
    return job

@router.post("/jobs/{job_id}/rerun")
def rerun_job(job_id: int, session: Session = Depends(get_session)):
    job = session.get(AnalysisJob, job_id)
    if not job:
        raise HTTPException(status_code=404, detail="Job not found")

    for model in [DetectionResult, ActionResult, StepEvent, Violation, EvaluationReport]:
        rows = session.exec(select(model).where(model.job_id == job_id)).all()
        for row in rows:
            session.delete(row)
    job.status = AnalysisStatus.pending
    job.score = 0
    job.summary = ""
    job.error_message = ""
    session.add(job)
    session.commit()
    run_analysis(job_id)
    session.refresh(job)
    return job

@router.get("/jobs/{job_id}/detail")
def get_analysis_detail(job_id: int, session: Session = Depends(get_session)):
    job = session.get(AnalysisJob, job_id)
    if not job:
        raise HTTPException(status_code=404, detail="Job not found")
    submission = session.get(VideoSubmission, job.submission_id)
    return {
        "job": job,
        "submission": submission,
        "detections": session.exec(select(DetectionResult).where(DetectionResult.job_id ==
job_id).order_by(ActionResult.start_sec)).all(),
        "actions": session.exec(select(ActionResult).where(ActionResult.job_id ==
job_id).order_by(ActionResult.start_sec)).all(),
        "steps": session.exec(select(StepEvent).where(StepEvent.job_id ==
job_id).order_by(StepEvent.start_sec)).all(),
        "violations": session.exec(select(Violation).where(Violation.job_id ==
job_id).order_by(Violation.deduction.desc())).all(),
        "report": session.exec(select(EvaluationReport).where(EvaluationReport.job_id == job_id)).first(),
    }

@router.get("/stats/overview")
def overview(session: Session = Depends(get_session)):
    jobs = session.exec(select(AnalysisJob)).all()
    completed = [job for job in jobs if job.status == AnalysisStatus.completed]
    average = round(sum(job.score for job in completed) / len(completed), 1) if completed else 0
    return {
        "total_jobs": len(jobs),
        "completed_jobs": len(completed),
        "average_score": average,
        "excellent_count": len([job for job in completed if job.score >= 90]),
        "risk_count": len([job for job in completed if job.score < 70]),
    }

```

backend/app/routers/catalog.py

```

from fastapi import APIRouter, Depends
from sqlmodel import Session, select

from ..database import get_session
from ..models import KnowledgeDocument, ScoringRule, StandardStep

router = APIRouter(prefix="/api/catalog", tags=["catalog"])

```

```

@router.get("/steps")
def list_steps(session: Session = Depends(get_session)):
    return session.exec(select(StandardStep).order_by(StandardStep.order_index)).all()

@router.get("/rules")
def list_rules(session: Session = Depends(get_session)):
    return session.exec(select(ScoringRule).order_by(ScoringRule.id)).all()

@router.get("/knowledge")
def list_knowledge(session: Session = Depends(get_session)):
    return session.exec(select(KnowledgeDocument).order_by(KnowledgeDocument.id)).all()

```

backend/app/routers/submissions.py

```

from pathlib import Path
from uuid import uuid4

from fastapi import APIRouter, BackgroundTasks, Depends, File, Form, HTTPException, UploadFile
from fastapi.responses import FileResponse
from sqlmodel import Session, select

from ..database import UPLOAD_DIR, get_session
from ..models import AnalysisJob, VideoSubmission
from .analysis import run_analysis

router = APIRouter(prefix="/api/submissions", tags=["submissions"])

@router.get("")
def list_submissions(task_id: int | None = None, session: Session = Depends(get_session)):
    query = select(VideoSubmission).order_by(VideoSubmission.uploaded_at.desc())
    if task_id:
        query = select(VideoSubmission).where(VideoSubmission.task_id == task_id).order_by(VideoSubmission.uploaded_at.desc())
    return session.exec(query).all()

@router.post("")
async def upload_submission(
    background_tasks: BackgroundTasks,
    task_id: int = Form(...),
    student_name: str = Form(...),
    student_no: str = Form(""),
    file: UploadFile = File(...),
    session: Session = Depends(get_session),
):
    suffix = Path(file.filename or "video.mp4").suffix or ".mp4"
    stored_filename = f"{uuid4().hex}{suffix}"
    target = UPLOAD_DIR / stored_filename
    content = await file.read()
    target.write_bytes(content)

    submission = VideoSubmission(
        task_id=task_id,
        student_name=student_name,
        student_no=student_no,
        original_filename=file.filename or stored_filename,
        stored_filename=stored_filename,
        content_type=file.content_type or "",
        size_bytes=len(content),
    )
    session.add(submission)
    session.commit()
    session.refresh(submission)

    job = AnalysisJob(submission_id=submission.id)
    session.add(job)

```

```

    session.commit()
    session.refresh(job)
    background_tasks.add_task(run_analysis, job.id)

    return {"submission": submission, "job": job}

@router.get("/{submission_id}/video")
def get_video(submission_id: int, session: Session = Depends(get_session)):
    submission = session.get(VideoSubmission, submission_id)
    if not submission:
        raise HTTPException(status_code=404, detail="Submission not found")
    path = UPLOAD_DIR / submission.stored_filename
    if not path.exists():
        raise HTTPException(status_code=404, detail="Video file not found")
    return FileResponse(path, media_type=submission.content_type or "video/mp4",
        filename=submission.original_filename)

```

backend/app/routers/tasks.py

```

from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from sqlmodel import Session, select

from ..database import get_session
from ..models import TrainingTask

router = APIRouter(prefix="/api/tasks", tags=["tasks"])

class TaskCreate(BaseModel):
    title: str
    course: str
    class_name: str
    teacher: str
    description: str = ""

@router.get("")
def list_tasks(session: Session = Depends(get_session)):
    return session.exec(select(TrainingTask).order_by(TrainingTask.created_at.desc())).all()

@router.post("")
def create_task(payload: TaskCreate, session: Session = Depends(get_session)):
    task = TrainingTask(**payload.model_dump())
    session.add(task)
    session.commit()
    session.refresh(task)
    return task

@router.get("/{task_id}")
def get_task(task_id: int, session: Session = Depends(get_session)):
    task = session.get(TrainingTask, task_id)
    if not task:
        raise HTTPException(status_code=404, detail="Task not found")
    return task

```

backend/app/seed.py

```

from sqlmodel import Session, select

from .models import KnowledgeDocument, ScoringRule, StandardStep

DEFAULT_STEPS = [
    ("ppe", "穿戴防护用品", "佩戴安全帽、绝缘手套、绝缘靴等个人防护用品。"),

```

```

("ticket", "确认工作票/操作票", "核对作业任务、人员、范围和安全措施。"),
("tool_check", "检查工具", "检查验电器、接地线、操作杆等工具状态。"),
("power_off", "停电确认", "确认设备处于停电状态并完成安全确认。"),
("voltage_test", "验电", "使用合格验电器对目标设备进行验电。"),
("ground_wire", "挂接地线", "验电后按规定装设接地线。"),
("switch_operation", "操作开关/隔离开关", "按操作票顺序完成开关、刀闸或隔离开关操作。"),
("review", "复核状态", "复核设备状态、接地线状态和现场安全状态。"),
("cleanup", "清理现场", "清点工具、撤离人员、恢复现场秩序。"),
]

DEFAULT_RULES = [
    ("missing_ppe", "不完整穿戴防护用品", "detection_required", "ppe", 15, "high", "进入作业区域前必须完成个人防护用品穿戴。"),
    ("missing_ticket", "未确认工作票/操作票", "step_required", "ticket", 10, "medium", "操作前应核对工作票或操作票。"),
    ("missing_voltage_test", "缺失验电步骤", "step_required", "voltage_test", 20, "critical", "挂接地线和设备操作前必须验电。"),
    ("missing_ground_wire", "缺失挂接地线步骤", "step_required", "ground_wire", 20, "critical", "验电后应按规定挂接地线。"),
    ("wrong_sequence_ground_before_test", "挂接地线早于验电", "sequence", "voltage_test>ground_wire", 25, "critical", "挂接地线必须发生在验电之后。"),
    ("missing_review", "未复核设备状态", "step_required", "review", 8, "medium", "操作完成后应复核设备和现场状态。"),
]

DEFAULT_DOCS = [
    (
        "铁道供电作业评分总则",
        "评分细则",
        "评价应以视频识别事实为基础，规则评分为硬性依据，教师复核为最终确认。安全防护、验电、接地线、操作顺序属于重点扣分项。",
    ),
    (
        "停电验电接地操作要求",
        "作业规程",
        "作业人员应先确认停电，再完成验电。确认无电后方可装设接地线，严禁未验电或验电前挂接地线。",
    ),
    (
        "个人防护用品要求",
        "安全规范",
        "进入铁道供电实训操作区域前，应按要求佩戴安全帽、绝缘手套、绝缘靴等防护用品。",
    ),
]

def seed_defaults(session: Session) -> None:
    if not session.exec(select(StandardStep)).first():
        for index, (code, name, description) in enumerate(DEFAULT_STEPS, start=1):
            session.add(StandardStep(code=code, name=name, order_index=index, description=description))

    if not session.exec(select(ScoringRule)).first():
        for code, title, rule_type, target_code, deduction, severity, description in DEFAULT_RULES:
            session.add(
                ScoringRule(
                    code=code,
                    title=title,
                    rule_type=rule_type,
                    target_code=target_code,
                    deduction=deduction,
                    severity=severity,
                    description=description,
                )
            )

    if not session.exec(select(KnowledgeDocument)).first():
        for title, category, content in DEFAULT_DOCS:
            session.add(KnowledgeDocument(title=title, category=category, content=content))

    session.commit()

```

backend/app/services/ai/__init__.py

```
from .pipeline import RailwayPowerVideoAnalyzer

__all__ = ["RailwayPowerVideoAnalyzer"]
```

backend/app/services/ai/action_recognizer.py

```
from pathlib import Path
from typing import Any

from .config import AISettings
from .contracts import ActionFact

class MMActionRecognizer:
    def __init__(self, settings: AISettings):
        self.settings = settings
        self.recognizer: Any | None = None
        self.inference_recognizer: Any | None = None
        self.error: str | None = None
        if not settings.mmaction_enabled:
            self.error = "MMAction2 未启用或未配置 MMACTION_CONFIG/MMACTION_CHECKPOINT"
            return
        try:
            from mmaction.apis import inference_recognizer, init_recognizer

            self.recognizer = init_recognizer(settings.mmaction_config, settings.mmaction_checkpoint,
device="cuda:0")
            self.inference_recognizer = inference_recognizer
        except Exception as exc:
            self.error = f"MMAction2 初始化失败: {exc}"

    def recognize(self, video_path: Path) -> tuple[list[ActionFact], list[str]]:
        if not self.recognizer or not self.inference_recognizer:
            return [], [self.error or "MMAction2 recognizer unavailable"]

        try:
            result = self.inference_recognizer(self.recognizer, str(video_path))
        except Exception as exc:
            return [], [f"MMAction2 推理失败: {exc}"]

        predictions = self._normalize_result(result)
        actions = [
            ActionFact(action=label, confidence=score, start_sec=index * 8.0, end_sec=index * 8.0 + 6.0)
            for index, (label, score) in enumerate(predictions[:5])
        ]
        return actions, []

    @staticmethod
    def _normalize_result(result: Any) -> list[tuple[str, float]]:
        pred_scores = getattr(result, "pred_score", None)
        if pred_scores is None:
            return []
        values = pred_scores.tolist()
        ranked = sorted(enumerate(values), key=lambda item: item[1], reverse=True)
        return [(f"action_{index}", float(score)) for index, score in ranked]
```

backend/app/services/ai/config.py

```
from dataclasses import dataclass
import os
from pathlib import Path

def _bool_env(name: str, default: bool = False) -> bool:
    value = os.getenv(name)
```

```

    if value is None:
        return default
    return value.strip().lower() in {"1", "true", "yes", "on"}

@dataclass(frozen=True)
class AISettings:
    provider: str = os.getenv("AI_ANALYZER_PROVIDER", "mock")
    frame_sample_fps: float = float(os.getenv("AI_FRAME_SAMPLE_FPS", "0.2"))
    max_frames: int = int(os.getenv("AI_MAX_FRAMES", "12"))
    yolo_model_path: str = os.getenv("YOLO_MODEL_PATH", "")
    yolo_confidence: float = float(os.getenv("YOLO_CONFIDENCE", "0.35"))
    mediapipe_enabled: bool = _bool_env("MEDIAPIPE_ENABLED", False)
    mmaction_config: str = os.getenv("MMACTION_CONFIG", "")
    mmaction_checkpoint: str = os.getenv("MMACTION_CHECKPOINT", "")
    multimodal_provider: str = os.getenv("MULTIMODAL_PROVIDER", "none")
    openai_model: str = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")
    openai_api_key: str = os.getenv("OPENAI_API_KEY", "")

    @property
    def yolo_enabled(self) -> bool:
        return self.provider in {"yolo", "hybrid", "full"} and bool(self.yolo_model_path)

    @property
    def mmaction_enabled(self) -> bool:
        return self.provider in {"mmaction", "hybrid", "full"} and bool(self.mmaction_config and
self.mmaction_checkpoint)

def model_path(value: str) -> Path | None:
    return Path(value).expanduser().resolve() if value else None

```

backend/app/services/ai/contracts.py

```

from dataclasses import dataclass
from pathlib import Path

```

```

@dataclass
class VideoFrame:
    path: Path
    timestamp_sec: float

```

```

@dataclass
class DetectionFact:
    label: str
    confidence: float
    timestamp_sec: float
    bbox: str = ""

```

```

@dataclass
class PoseFact:
    timestamp_sec: float
    confidence: float
    landmarks_json: str

```

```

@dataclass
class ActionFact:
    action: str
    confidence: float
    start_sec: float
    end_sec: float

```

```

@dataclass
class StepFact:

```

```
step_code: str
step_name: str
start_sec: float
end_sec: float
confidence: float
evidence: str
```

```
@dataclass
class AnalysisFacts:
    detections: List[DetectionFact]
    actions: List[ActionFact]
    steps: List[StepFact]
    notes: List[str]
```

backend/app/services/ai/frame_extractor.py

```
from pathlib import Path
import shutil
import subprocess
from tempfile import TemporaryDirectory

from .contracts import VideoFrame

class FrameExtractor:
    def __init__(self, sample_fps: float, max_frames: int):
        self.sample_fps = sample_fps
        self.max_frames = max_frames

    def extract(self, video_path: Path) -> tuple[TemporaryDirectory, List[VideoFrame]]:
        temp_dir = TemporaryDirectory(prefix="railway-ai-frames-")
        output_dir = Path(temp_dir.name)

        if not shutil.which("ffmpeg"):
            return temp_dir, []

        pattern = output_dir / "frame_%05d.jpg"
        command = [
            "ffmpeg",
            "-hide_banner",
            "-loglevel",
            "error",
            "-i",
            str(video_path),
            "-vf",
            f"fps={self.sample_fps}",
            "-frames:v",
            str(self.max_frames),
            str(pattern),
        ]
        result = subprocess.run(command, capture_output=True, text=True, check=False)
        if result.returncode != 0:
            return temp_dir, []

        interval = 1 / self.sample_fps if self.sample_fps > 0 else 5
        frames = [
            VideoFrame(path=path, timestamp_sec=index * interval)
            for index, path in enumerate(sorted(output_dir.glob("frame_*.jpg")))
        ]
        return temp_dir, frames
```

backend/app/services/ai/mock_analyzer.py

```
from pathlib import Path

from .contracts import ActionFact, AnalysisFacts, DetectionFact, StepFact
from .steps import STEP_NAMES
```

```

class MockAnalyzer:
    def analyze(self, video_path: Path, original_filename: str) -> AnalysisFacts:
        marker = original_filename.lower()
        is_bad = any(word in marker for word in ["bad", "error", "违规", "错误"])
        no_glove = is_bad or any(word in marker for word in ["no-glove", "noglove", "无手套"])
        skip_ground = any(word in marker for word in ["skip-ground", "noground", "未接地"])
        wrong_order = any(word in marker for word in ["wrong-order", "顺序错误"])

        detections = [
            DetectionFact("person", 0.98, 3.0, "0.20,0.12,0.42,0.86"),
            DetectionFact("safety_helmet", 0.94, 4.0, "0.25,0.05,0.38,0.18"),
            DetectionFact("voltage_detector", 0.88, 34.0, "0.58,0.22,0.68,0.48"),
            DetectionFact("ground_wire", 0.84, 50.0, "0.48,0.42,0.75,0.66"),
            DetectionFact("switch", 0.91, 68.0, "0.70,0.16,0.90,0.50"),
        ]
        if not no_glove:
            detections.append(DetectionFact("insulating_gloves", 0.92, 5.0, "0.30,0.32,0.45,0.52"))

        ordered_codes = [
            "ppe",
            "ticket",
            "tool_check",
            "power_off",
            "voltage_test",
            "ground_wire",
            "switch_operation",
            "review",
            "cleanup",
        ]
        if skip_ground and "ground_wire" in ordered_codes:
            ordered_codes.remove("ground_wire")
        if wrong_order:
            ordered_codes = [
                "ppe",
                "ticket",
                "tool_check",
                "power_off",
                "ground_wire",
                "voltage_test",
                "switch_operation",
                "review",
                "cleanup",
            ]
        if is_bad and "review" in ordered_codes:
            ordered_codes.remove("review")

        steps: list[StepFact] = []
        actions: list>ActionFact = []
        for index, code in enumerate(ordered_codes):
            start = 6.0 + index * 10.0
            end = start + 6.0
            confidence = 0.90 if code in {"ppe", "voltage_test", "ground_wire"} else 0.82
            steps.append(
                StepFact(
                    step_code=code,
                    step_name=STEP_NAMES[code],
                    start_sec=start,
                    end_sec=end,
                    confidence=confidence,
                    evidence=f"在 {_format_time(start)} 到 {_format_time(end)} 识别到 “{STEP_NAMES[code]}” 相关动
作。",
                )
            )
            actions.append(ActionFact(STEP_NAMES[code], confidence, start, end))

        return AnalysisFacts(detections=detections, actions=actions, steps=steps, notes=["使用 mock 分析器生成演示结果"])

```

```
def _format_time(seconds: float) -> str:
    minute = int(seconds // 60)
    second = int(seconds % 60)
    return f"{minute:02d}:{second:02d}"
```

backend/app/services/ai/multimodal.py

```
from .config import AISettings
from .contracts import ActionFact, DetectionFact, StepFact, VideoFrame

class MultimodalExplainer:
    def __init__(self, settings: AISettings):
        self.settings = settings

    def enrich_steps(
        self,
        frames: List[VideoFrame],
        detections: List[DetectionFact],
        actions: List[ActionFact],
        steps: List[StepFact],
    ) -> tuple[List[StepFact], List[str]]:
        if self.settings.multimodal_provider == "openai":
            return self._openai_enrich(frames, detections, actions, steps)
        return steps, ["多模态解释未启用, 使用规则模板生成证据说明"]

    def _openai_enrich(
        self,
        frames: List[VideoFrame],
        detections: List[DetectionFact],
        actions: List[ActionFact],
        steps: List[StepFact],
    ) -> tuple[List[StepFact], List[str]]:
        if not self.settings.openai_api_key:
            return steps, ["OpenAI 多模态解释未配置 OPENAI_API_KEY, 跳过"]
        try:
            from openai import OpenAI

            client = OpenAI(api_key=self.settings.openai_api_key)
            prompt = (
                "你是铁道供电作业实训考评助手。请基于检测对象、动作识别和步骤事件，"
                "生成简短、可审计的中文证据说明。不要新增没有证据支持的事实。"
            )
            summary = {
                "detections": [item.__dict__ for item in detections[:20]],
                "actions": [item.__dict__ for item in actions[:10]],
                "steps": [item.__dict__ for item in steps],
                "sampled_frames": [str(frame.path.name) for frame in frames[:4]],
            }
            response = client.responses.create(
                model=self.settings.openai_model,
                input=[
                    {"role": "system", "content": prompt},
                    {"role": "user", "content": f"结构化证据: {summary}"},
                ],
            )
            text = getattr(response, "output_text", "")
            if text and steps:
                steps[0].evidence = f"{steps[0].evidence} 多模态说明: {text[:180]}"
            return steps, ["已调用 OpenAI 多模态解释生成证据说明"]
        except Exception as exc:
            return steps, [f"OpenAI 多模态解释失败, 已回退模板说明: {exc}"]
```

backend/app/services/ai/pipeline.py

```
from pathlib import Path

from .action_recognizer import MMActionRecognizer
```

```

from .config import AISettings
from .contracts import ActionFact, AnalysisFacts, DetectionFact, StepFact
from .frame_extractor import FrameExtractor
from .mock_analyzer import MockAnalyzer
from .multimodal import MultimodalExplainer
from .pose_estimator import MediaPipePoseEstimator
from .steps import ACTION_TO_STEP, OBJECT_TO_STEP, STEP_NAMES
from .yolo_detector import YoloDetector

class RailwayPowerVideoAnalyzer:
    def __init__(self, settings: AISettings | None = None):
        self.settings = settings or AISettings()
        self.mock = MockAnalyzer()
        self.frame_extractor = FrameExtractor(self.settings.frame_sample_fps, self.settings.max_frames)
        self.yolo = YoloDetector(self.settings)
        self.pose = MediaPipePoseEstimator(self.settings)
        self.actions = MMActionRecognizer(self.settings)
        self.explainer = MultimodalExplainer(self.settings)

    def analyze(self, video_path: Path, original_filename: str) -> AnalysisFacts:
        if self.settings.provider == "mock":
            return self.mock.analyze(video_path, original_filename)

        temp_dir, frames = self.frame_extractor.extract(video_path)
        notes: list[str] = []
        try:
            detections, detection_notes = self.yolo.detect(frames)
            notes.extend(detection_notes)

            _, pose_notes = self.pose.estimate(frames)
            notes.extend(pose_notes)

            actions, action_notes = self.actions.recognize(video_path)
            notes.extend(action_notes)

            if not detections and not actions:
                fallback = self.mock.analyze(video_path, original_filename)
                fallback.notes.extend(["真实模型未产出可评分事实, 已回退 mock 分析"] + notes)
                return fallback

            steps = self._build_steps(detections, actions)
            if not steps:
                fallback = self.mock.analyze(video_path, original_filename)
                fallback.detections = detections or fallback.detections
                fallback.actions = actions or fallback.actions
                fallback.notes.extend(["模型事实无法映射标准步骤, 已回退 mock 步骤"] + notes)
                return fallback

            steps, explain_notes = self.explainer.enrich_steps(frames, detections, actions, steps)
            notes.extend(explain_notes)
            return AnalysisFacts(detections=detections, actions=actions, steps=steps, notes=notes)
        finally:
            temp_dir.cleanup()

    def _build_steps(self, detections: list[DetectionFact], actions: list>ActionFact]) -> list[StepFact]:
        events: dict[str, tuple[float, float, float, str]] = {}

        for detection in detections:
            step_code = OBJECT_TO_STEP.get(detection.label)
            if not step_code:
                continue
            current = events.get(step_code)
            evidence = f"[_format_time(detection.timestamp_sec)] 检测到 {detection.label}, 置信度 {detection.confidence:.2f}。"
            if current is None or detection.timestamp_sec < current[0]:
                events[step_code] = (detection.timestamp_sec, detection.timestamp_sec + 3.0, detection.confidence,
evidence)

        for action in actions:
            step_code = ACTION_TO_STEP.get(action.action, action.action if action.action in STEP_NAMES else "")

```

```

        if not step_code:
            continue
        current = events.get(step_code)
        evidence = f"{_format_time(action.start_sec)} 到 {_format_time(action.end_sec)} 识别到动作
“{action.action}”。"
        if current is None or action.confidence > current[2]:
            events[step_code] = (action.start_sec, action.end_sec, action.confidence, evidence)

    steps = [
        StepFact(
            step_code=code,
            step_name=STEP_NAMES[code],
            start_sec=value[0],
            end_sec=value[1],
            confidence=value[2],
            evidence=value[3],
        )
        for code, value in events.items()
        if code in STEP_NAMES
    ]
    return sorted(steps, key=lambda item: item.start_sec)

def _format_time(seconds: float) -> str:
    minute = int(seconds // 60)
    second = int(seconds % 60)
    return f"{minute:02d}:{second:02d}"

```

backend/app/services/ai/pose_estimator.py

```

import json
from typing import Any

from .config import AISettings
from .contracts import PoseFact, VideoFrame

class MediaPipePoseEstimator:
    def __init__(self, settings: AISettings):
        self.settings = settings
        self.detector: Any | None = None
        self.error: str | None = None
        if not settings.mediapipe_enabled:
            self.error = "MediaPipe 未启用"
            return
        try:
            import mediapipe as mp

            self.mp = mp
            self.detector = mp.solutions.pose.Pose(static_image_mode=True, model_complexity=1)
        except Exception as exc:
            self.error = f"MediaPipe 初始化失败: {exc}"

    def estimate(self, frames: list[VideoFrame]) -> tuple[list[PoseFact], list[str]]:
        if not self.detector:
            return [], [self.error or "MediaPipe pose unavailable"]

        try:
            import cv2
        except Exception as exc:
            return [], [f"OpenCV 不可用, 跳过 MediaPipe 姿态识别: {exc}"]

        poses: list[PoseFact] = []
        notes: list[str] = []
        for frame in frames:
            image = cv2.imread(str(frame.path))
            if image is None:
                continue
            rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

```

```

        result = self.detector.process(rgb)
        if not result.pose_landmarks:
            continue
        landmarks = [
            {"x": item.x, "y": item.y, "z": item.z, "visibility": item.visibility}
            for item in result.pose_landmarks.landmark
        ]
        visibility = sum(item["visibility"] for item in landmarks) / len(landmarks)
        poses.append(PoseFact(frame.timestamp_sec, float(visibility), json.dumps(landmarks,
ensure_ascii=False)))
    return poses, notes

```

backend/app/services/ai/steps.py

```

STEP_NAMES = {
    "ppe": "穿戴防护用品",
    "ticket": "确认工作票/操作票",
    "tool_check": "检查工器具",
    "power_off": "停电确认",
    "voltage_test": "验电",
    "ground_wire": "挂接地线",
    "switch_operation": "操作开关/隔离开关",
    "review": "复核状态",
    "cleanup": "清理现场",
}

OBJECT_TO_STEP = {
    "safety_helmet": "ppe",
    "insulating_gloves": "ppe",
    "insulating_boots": "ppe",
    "work_ticket": "ticket",
    "operation_ticket": "ticket",
    "voltage_detector": "voltage_test",
    "ground_wire": "ground_wire",
    "switch": "switch_operation",
    "disconnect_switch": "switch_operation",
    "circuit_breaker": "switch_operation",
}

ACTION_TO_STEP = {
    "穿戴防护用品": "ppe",
    "确认工作票/操作票": "ticket",
    "检查工器具": "tool_check",
    "停电确认": "power_off",
    "验电": "voltage_test",
    "挂接地线": "ground_wire",
    "操作开关/隔离开关": "switch_operation",
    "复核状态": "review",
    "清理现场": "cleanup",
}

```

backend/app/services/ai/yolo_detector.py

```

from pathlib import Path
from typing import Any

from .config import AISettings, model_path
from .contracts import DetectionFact, VideoFrame

class YoloDetector:
    def __init__(self, settings: AISettings):
        self.settings = settings
        self.model: Any | None = None
        self.error: str | None = None
        if not settings.yolo_enabled:

```

```

        self.error = "YOLO 未启用或未配置 YOLO_MODEL_PATH"
        return
    try:
        from ultralytics import YOLO

        path = model_path(settings.yolo_model_path)
        self.model = YOLO(str(path))
    except Exception as exc:
        self.error = f"YOLO 初始化失败: {exc}"

def detect(self, frames: list[VideoFrame]) -> tuple[list[DetectionFact], list[str]]:
    if not self.model:
        return [], [self.error or "YOLO detector unavailable"]

    detections: list[DetectionFact] = []
    notes: list[str] = []
    for frame in frames:
        try:
            results = self.model.predict(str(frame.path), conf=self.settings.yolo_confidence, verbose=False)
        except Exception as exc:
            notes.append(f"YOLO 推理失败 {frame.path.name}: {exc}")
            continue

        for result in results:
            names = result.names
            boxes = getattr(result, "boxes", None)
            if boxes is None:
                continue
            for box in boxes:
                cls_id = int(box.cls[0])
                conf = float(box.conf[0])
                xyxy = [float(value) for value in box.xyxy[0].tolist()]
                label = str(names.get(cls_id, cls_id))
                detections.append(
                    DetectionFact(
                        label=label,
                        confidence=conf,
                        timestamp_sec=frame.timestamp_sec,
                        bbox=",".join(f"{value:.4f}" for value in xyxy),
                    )
                )
    return detections, notes

```

backend/app/services/analyzer.py

```

from pathlib import Path

from .ai.contracts import ActionFact, AnalysisFacts, DetectionFact, StepFact
from .ai.pipeline import RailwayPowerVideoAnalyzer

class Analyzer:
    """Compatibility facade used by the existing analysis job router."""

    def __init__(self):
        self.pipeline = RailwayPowerVideoAnalyzer()

    def analyze(self, video_path: Path, original_filename: str) -> AnalysisFacts:
        return self.pipeline.analyze(video_path, original_filename)

__all__ = ["Analyzer", "ActionFact", "AnalysisFacts", "DetectionFact", "StepFact"]

```

backend/app/services/report.py

```

from sqlmodel import Session, select

from ..models import EvaluationReport, KnowledgeDocument, StepEvent, Violation

```

```

def generate_report(session: Session, job_id: int, score: float) -> EvaluationReport:
    steps = session.exec(select(StepEvent).where(StepEvent.job_id == job_id).order_by(StepEvent.start_sec)).all()
    violations = session.exec(select(Violation).where(Violation.job_id ==
job_id).order_by(Violation.deduction.desc())).all()
    docs = session.exec(select(KnowledgeDocument).limit(3)).all()

    step_names = ", ".join(step.step_name for step in steps) or "未识别到有效操作步骤"
    doc_titles = ", ".join(doc.title for doc in docs)

    if score >= 90:
        conclusion = "本次作业流程整体规范，关键安全步骤识别完整，建议作为合格示范视频归档。"
    elif score >= 70:
        conclusion = "本次作业基本完成，但存在需要教师复核和学生整改的扣分项。"
    else:
        conclusion = "本次作业存在较明显安全或流程问题，建议重新训练并复评。"

    strengths = f"系统识别到的操作流程包括：{step_names}。评价参考知识库依据：{doc_titles}。"
    if violations:
        problems = "; ".join(f"{item.title} (扣 {item.deduction:g} 分): {item.reason}" for item in violations)
        suggestions = "; ".join(item.suggestion for item in violations)
    else:
        problems = "未发现硬性规则扣分项。"
        suggestions = "保持作业前防护检查、操作中步骤口述和操作后复核的完整视频证据。"

    return EvaluationReport(
        job_id=job_id,
        score=score,
        conclusion=conclusion,
        strengths=strengths,
        problems=problems,
        suggestions=suggestions,
    )

```

backend/app/services/rule_engine.py

```

from sqlmodel import Session, select

from ..models import DetectionResult, ScoringRule, StepEvent, Violation

PPE_LABELS = {"safety_helmet", "insulating_gloves"}

def evaluate_rules(session: Session, job_id: int) -> tuple[float, list[Violation]]:
    rules = session.exec(select(ScoringRule).order_by(ScoringRule.id)).all()
    steps = session.exec(select(StepEvent).where(StepEvent.job_id == job_id)).all()
    detections = session.exec(select(DetectionResult).where(DetectionResult.job_id == job_id)).all()

    step_by_code = {step.step_code: step for step in steps}
    detected_labels = {item.label for item in detections}
    violations: list[Violation] = []

    for rule in rules:
        violation = None
        if rule.rule_type == "detection_required" and rule.target_code == "ppe":
            missing = sorted(PPE_LABELS - detected_labels)
            if missing:
                violation = Violation(
                    job_id=job_id,
                    rule_code=rule.code,
                    title=rule.title,
                    deduction=rule.deduction,
                    severity=rule.severity,
                    timestamp_sec=0,
                    reason=f"视频证据未完整识别到防护用品：{' '.join(missing)}。",
                    suggestion="进入作业区域前完成安全帽、绝缘手套、绝缘靴等防护用品检查，并在视频中清晰展示。",
                )
    )

```

```

elif rule.rule_type == "step_required":
    if rule.target_code not in step_by_code:
        violation = Violation(
            job_id=job_id,
            rule_code=rule.code,
            title=rule.title,
            deduction=rule.deduction,
            severity=rule.severity,
            timestamp_sec=None,
            reason=f"未识别到必需步骤: {rule.description}",
            suggestion="按标准流程补充该步骤, 并保证动作完整、可见、可复核。",
        )
    elif rule.rule_type == "sequence":
        before_code, after_code = rule.target_code.split(">", 1)
        before = step_by_code.get(before_code)
        after = step_by_code.get(after_code)
        if before and after and before.start_sec > after.start_sec:
            violation = Violation(
                job_id=job_id,
                rule_code=rule.code,
                title=rule.title,
                deduction=rule.deduction,
                severity=rule.severity,
                timestamp_sec=after.start_sec,
                reason=f"识别到 “{after.step_name}” 发生在 “{before.step_name}” 之前, 不符合操作顺序。",
                suggestion=rule.description,
            )

if violation:
    session.add(violation)
    violations.append(violation)

score = max(0.0, 100.0 - sum(item.deduction for item in violations))
return score, violations

```

backend/requirements.txt

```

fastapi==0.115.6
uvicorn[standard]==0.32.1
sqlmodel==0.0.22
python-multipart==0.0.19
pydantic==2.10.3

# Optional real AI backends. Install only on machines that need model inference.
# ultralytics>=8.3.0
# opencv-python>=4.10.0
# mediapipe>=0.10.14
# openai>=1.59.0
# MMAction2 is usually installed from its official docs with matching PyTorch/CUDA.

```

backend/smoke_test.py

```

from pathlib import Path

from fastapi.testclient import TestClient

from app.main import app

def main() -> None:
    with TestClient(app) as client:
        health = client.get("/api/health")
        assert health.status_code == 200, health.text

        steps = client.get("/api/catalog/steps")
        assert steps.status_code == 200, steps.text
        assert len(steps.json()) >= 9

```

```

task_payload = {
    "title": "冒烟测试-接触网停电验电接地",
    "course": "铁道供电安全实训",
    "class_name": "冒烟测试班",
    "teacher": "系统测试",
    "description": "用于验证创建任务、上传视频、自动分析和报告生成。",
}
task = client.post("/api/tasks", json=task_payload)
assert task.status_code == 200, task.text
task_id = task.json()["id"]

sample = Path("data/uploads/smoke-no-glove.txt")
sample.parent.mkdir(parents=True, exist_ok=True)
sample.write_text("fake video bytes for smoke test", encoding="utf-8")

with sample.open("rb") as file:
    upload = client.post(
        "/api/submissions",
        data={"task_id": str(task_id), "student_name": "冒烟学生", "student_no": "SMOKE001"},
        files={"file": ("smoke-no-glove.mp4", file, "video/mp4")},
    )
assert upload.status_code == 200, upload.text
job_id = upload.json()["job"]["id"]

detail = client.get(f"/api/analysis/jobs/{job_id}/detail")
assert detail.status_code == 200, detail.text
payload = detail.json()
assert payload["job"]["status"] == "completed", payload["job"]
assert payload["report"]["score"] < 100
assert payload["violations"], "Expected no-glove violation"
print(
    {
        "health": health.json()["status"],
        "steps": len(steps.json()),
        "task_id": task_id,
        "job_id": job_id,
        "status": payload["job"]["status"],
        "score": payload["report"]["score"],
        "violations": [item["title"] for item in payload["violations"]],
    }
)

if __name__ == "__main__":
    main()

```

frontend/index.html

```

<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>铁道供电作业视频智能评价平台</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>

```

frontend/package-lock.json

```

{
  "name": "railway-power-operation-video-ai-evaluator",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,

```

```

"packages": {
  "": {
    "name": "railway-power-operation-video-ai-evaluator",
    "version": "1.0.0",
    "dependencies": {
      "@vitejs/plugin-vue": "^5.2.1",
      "vite": "^6.0.3",
      "vue": "^3.5.13"
    },
    "devDependencies": {}
  },
  "node_modules/@babel/helper-string-parser": {
    "version": "7.29.7",
    "resolved": "****",
    "integrity": "sha512-Pb5ijPrZ89GDH8223L4UP8i6QApWxs04RbPQJTeWDV0/keR2E36MeKnyr6LYmUUvqRRI+Iv87SuF1W6ErINzYw==",
    "license": "MIT",
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/helper-validator-identifier": {
    "version": "7.29.7",
    "resolved": "****",
    "integrity": "sha512-qehxGkRj55h/ff8EMaJ+cYhyaKLHIXqYDn682wQD7RNp9Uuj0QsHog2uS0r2vzr4pW+sXf90NeeayjcNaX3fFg==",
    "license": "MIT",
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/parser": {
    "version": "7.29.7",
    "resolved": "****",
    "integrity": "sha512-hn0RnjP/1P/zFEndoeX+n+t1RwWRJiJpM/j07FW32Kn9r5+sJB2JW0dYo4L6k78j15eCwY3Gm/7364B1EMwtNg==",
    "license": "MIT",
    "dependencies": {
      "@babel/types": "^7.29.7"
    },
    "bin": {
      "parser": "bin/babel-parser.js"
    },
    "engines": {
      "node": ">=6.0.0"
    }
  },
  "node_modules/@babel/types": {
    "version": "7.29.7",
    "resolved": "****",
    "integrity": "sha512-4zBIxpPzowiZpusoFkyGVwakdRJUyuH5PxQ/PrqghfdFWWasvncdPfQXhrendai+gyLARuLZjZowC0j6fjT4pA==",
    "license": "MIT",
    "dependencies": {
      "@babel/helper-string-parser": "^7.29.7",
      "@babel/helper-validator-identifier": "^7.29.7"
    },
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@esbuild/aix-ppc64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-Hhmwd6CInZ3dwpUGTF8fJG6yoWmsToE+vYgD4nytZVxcu1uLHpUQRAB1UJ8+N1Am3Mz4+x0ByoQoSzf4D+CpkA==",
    "cpu": [
      "ppc64"
    ],
    "license": "MIT",
    "optional": true,

```

```

    "os": [
      "aix"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/android-arm": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-VJ+sKvNA/GE7Ccacc9Cha7bpS8nyzVv0jdVgwNDaR4gDMC/2TTRc33Ip8qrNYUcpk0HUT50Z0bUcNNVZQ9RLlg==",
    "cpu": [
      "arm"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "android"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/android-arm64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-6AAmLG7zwD1Z159jCKPvAxZd4y/VT00VkprYy+3N2FtJ8+BQWFXU+0xARIwA46c5tdD9SsKGZ/1ocqBS/gAKHg==",
    "cpu": [
      "arm64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "android"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/android-x64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-5jbb+2hhDHx5phYR2By8GTWEzn6I9UqR11Kwf22iKbNpYrsmRB18aX/9ivc5cabCUiAT/wM+YIZ6SG9Q06a8kg==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "android"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/darwin-arm64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-N3zl+LxHCifgILcMUP5016ESkeQjLj/959RxxNYIthIg+CQHInujFuXewbWMgnTo4cp5XVHqFPmpyu9J65C1Yg==",
    "cpu": [
      "arm64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "darwin"
    ]
  },

```

```

    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/darwin-x64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-
H9Q9ka4Kx21qHXwtLTUVbKJOAnmG1ipXhdWTmNXiPzPfWKpXqASVcWdnf2bnL73wgjNrFXAa3yYvBSd9pzfEIpA==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "darwin"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/freebsd-arm64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-
gA0Bx759+7Jve03K1S0vk0u5Lg/85dou3Ese0GUes8fLV0GxbhDDh/iZaok11Y8mtyKPGF3vP8XhnkDEAmzeg==",
    "cpu": [
      "arm64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "freebsd"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/freebsd-x64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-
TGB026Yw2xsHzxtbVFGEXBFH0FRAP7gtcPE7P5yP7wGy7cXK2o07Ry0hL5NLiqTLBh47XhmIUXGciXEYfYfBQ==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "freebsd"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-arm": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-
LPDGyC1JPDou8kGcywY0YILzWlhnnRjdof3UlcqYmS9EL818LLfJJc3PXXgZHrHCAKs/Z2SeZtDJr5Mrkxt0w==",
    "cpu": [
      "arm"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  }
}

```

```

    },
    "node_modules/@esbuild/linux-arm64": {
      "version": "0.25.12",
      "resolved": "****",
      "integrity": "sha512-8bwX7a8FghIgrupcxb4aUmYDLp8pX06rGh5HqDT7bB+8RdelLs6mHvrFHHW2JAOPZUbnjUpKTLg6ECyzvas2AQ==",
      "cpu": [
        "arm64"
      ],
      "license": "MIT",
      "optional": true,
      "os": [
        "linux"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/linux-ia32": {
      "version": "0.25.12",
      "resolved": "****",
      "integrity": "sha512-0y9KrdVnbMM2/vG8KfU0byhUN+EFCny9+8g202gYqSSVMonbsCfLjU0+rCci7pM0WBetz+oK/PIwHkzxkyharA==",
      "cpu": [
        "ia32"
      ],
      "license": "MIT",
      "optional": true,
      "os": [
        "linux"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/linux-loong64": {
      "version": "0.25.12",
      "resolved": "****",
      "integrity": "sha512-h//Lr5a9rib/v1GGqXVGzjL4TMvVTv+s1DPoxQdz7L/AYv6LDSxdIwzxkrPW438oUXiDtwM10o9PmwS/6Z0Ng==",
      "cpu": [
        "loong64"
      ],
      "license": "MIT",
      "optional": true,
      "os": [
        "linux"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/linux-mips64el": {
      "version": "0.25.12",
      "resolved": "****",
      "integrity": "sha512-iyRrM1Pzy9GFMdLsXn1iHUm18nhKnNMWscjmp4+hpafcZjrr2WbT//d20xaGLjXDBYHqRcl8HnxbX6uaA/eGVw==",
      "cpu": [
        "mips64el"
      ],
      "license": "MIT",
      "optional": true,
      "os": [
        "linux"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/linux-ppc64": {
      "version": "0.25.12",

```

```

    "resolved": "****",
    "integrity": "sha512-
9meM/LRXxMi5PSUqEXRCtVjEZBGwB7P/D4yT8UG/mwIdze2aV4Vo6U5gD3+RsoHXKkHCfSxZKzmDssVLRj1QQA==",
    "cpu": [
      "ppc64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-riscv64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-
Zr7KR4hgKUpWAwb1f3o5ygT04MzqVrGEGXGlnj15YQDJErYu/BGg+wmFLID0dJp0PmB0LLvxFIOXZgFRrdjR0w==",
    "cpu": [
      "riscv64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-s390x": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-
MsKnc0cgTNvdtiISc/jZs/Zf8d0cl/t3gYWX8J9ubBnV0wlk65UIEEvgBORTiLjloIwnBzLs4qhzPkJcitiZg==",
    "cpu": [
      "s390x"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/linux-x64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-
uqZMTLr/zR/ed4jIGnwSLkaHmPj0jJvnm6TVvitAa08SLS9Z0VM8wIRx7gWbJB5/J54YuIMInDquWYyVqLZkgw==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/netbsd-arm64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-
xXwcTq4GhRM7J9A8Gv5boanHhRa/Q9KLVmcyXHTaM4wKfIpWkdXiMog/KsnxzJOA1+nD+zoecuzqPmCRyBGjg==",

```

```
"cpu": [
  "arm64"
],
"license": "MIT",
"optional": true,
"os": [
  "netbsd"
],
"engines": {
  "node": ">=18"
}
},
"node_modules/@esbuild/netbsd-x64": {
  "version": "0.25.12",
  "resolved": "****",
  "integrity": "sha512-
Ld5pTLzPy3YwGec40uHh1aCVCRv0Xdh8DgRj fDy/oumVovmuSzWfnSJg+VtakB9Cm0gxN09BzWkj6mt01FMXkQ==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "netbsd"
  ],
  "engines": {
    "node": ">=18"
  }
},
"node_modules/@esbuild/openbsd-arm64": {
  "version": "0.25.12",
  "resolved": "****",
  "integrity": "sha512-
fF96T6KsBo/pkQI950FARU9apGNTSLZGsv1jZBA l cLL1MLj LNIWPBkj 5NLSz8aAzYKg+eNqknrUJ24QBybeR5A==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openbsd"
  ],
  "engines": {
    "node": ">=18"
  }
},
"node_modules/@esbuild/openbsd-x64": {
  "version": "0.25.12",
  "resolved": "****",
  "integrity": "sha512-
MZYXUkZHjQxUvzK7rN8DJ3SRmrVrke8ZyRusHLP+kuwqTcfWLyqMOE3sScPPyeIXN/mDJIfGXvcMqCgYKekoQw==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openbsd"
  ],
  "engines": {
    "node": ">=18"
  }
},
"node_modules/@esbuild/openharmony-arm64": {
  "version": "0.25.12",
  "resolved": "****",
  "integrity": "sha512-
rm0YWsQjncSXGA7Zv78Nbnw4XL6/dzr20cyrQf7ZmRcsovpCRBdhD43Nuk3y7XIow20xMVvwuRvk9XdASg==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openharmony"
  ],
  "engines": {
    "node": ">=18"
  }
}
```

```
    "license": "MIT",
    "optional": true,
    "os": [
      "openharmony"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/sunos-x64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-3wGSCDyuTHQUzt0nV7bocDy72r2LI33QL3gkDNGkod22EsYL04sMf0qLb8LuNKT0mgF/eDEDP5BFNwoBKH441w==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "sunos"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/win32-arm64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-rMmLrur64A7+DKlnSuwqUdRKyd3UE7oPJZmnljqEptesKM8wx9J8gx5u0+9Pq0fQQW8vqeKebwNXdf0yP+8Bsg==",
    "cpu": [
      "arm64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/win32-ia32": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-HkqnmBoCbCwxUKKNPBixiWDGCPqGVsrQfJoVGyLPT41XWF8LHuE5N6WhVia2n4o5QK5M4tYr21827fNhi4byQ==",
    "cpu": [
      "ia32"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@esbuild/win32-x64": {
    "version": "0.25.12",
    "resolved": "****",
    "integrity": "sha512-aLJC0uCZpTfRSL0CCDjcgLeBXPnCrEAhTBILpeAp7M/0FgoqtAetfBzX0xM00MusVVPpVjLPuMbREqnZCXaTnA==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
```

```

    "win32"
  ],
  "engines": {
    "node": ">=18"
  }
},
"node_modules/@jridgewell/sourcemap-codec": {
  "version": "1.5.5",
  "resolved": "****",
  "integrity": "sha512-cYQ9310grqxueWbl+WiUIAiUaDcj7W0q5fVhELjNVgRf0Uhy9fy2zTvfoqWsnebh8SL70VScFbICvJnLKB00g==",
  "license": "MIT"
},
"node_modules/@rollup/rollup-android-arm-eabi": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-F5QXMSiFebS9hKZj02XhWLLnRpJ3B3AR0P0tWbFBSj+6kCbg5m9j5JoHKd4mmSVy5mS/IMQLoYgYxCuJC0fxEQ==",
  "cpu": [
    "arm"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "android"
  ]
},
"node_modules/@rollup/rollup-android-arm64": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-GxxTKApUpzRhof7poWvCJHRF51C67u1R7D6Di luBE8wKU1u5GWE8t+v81JvJYtbawoBFX1hLv5Ei4eVjkWokaw==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "android"
  ]
},
"node_modules/@rollup/rollup-darwin-arm64": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-tua0TaJxMOB1ROVORS1jFZ/RpURFDJIOR2A6jWwQeawuFyS4gBW+rntLRaQd0EQ4bd6Vp44Z2rXW+YYDBsj6IA==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "darwin"
  ]
},
"node_modules/@rollup/rollup-darwin-x64": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-CSKq7MsP+5PFICydhAiR1K0UheI1A2jWxVKHPCBZ151yOutENwvnpocgVHkivu2kviURtCEB6zUQw0vs8RrhMg==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "darwin"
  ]
},
"node_modules/@rollup/rollup-freebsd-arm64": {
  "version": "4.60.4",

```

```
"resolved": "****
"integrity": "sha512-
+080kVdyvXMTJEciU2wS/pzm1IxntEEQx3z5TAVy4l32G0etZn+RsA48ARRrFm6Ri8fvqPQfgrvNxSjKAbnd3g==",
"cpu": [
  "arm64"
],
"license": "MIT",
"optional": true,
"os": [
  "freebsd"
]
},
"node_modules/@rollup/rollup-freebsd-x64": {
  "version": "4.60.4",
  "resolved": "****
"integrity": "sha512-
Iw3oMskH3AfNuhUOMSN7vNbd14me/NiYo2azqPz/Le16zHSa+3RRmLiCMWWQmh4LcndccU40xcJuTYJZxNo/Lw==",
"cpu": [
  "x64"
],
"license": "MIT",
"optional": true,
"os": [
  "freebsd"
]
},
"node_modules/@rollup/rollup-linux-arm-gnueabihf": {
  "version": "4.60.4",
  "resolved": "****
"integrity": "sha512-
EIPRXTVQpHyF8W0o219AD2yELtPehLTcTMz2fn6JsatLYSzf00hj3ruLF+yau0LF9/FtM2WpkT/hJh/KJFGhA==",
"cpu": [
  "arm"
],
"libc": [
  "glibc"
],
"license": "MIT",
"optional": true,
"os": [
  "linux"
]
},
"node_modules/@rollup/rollup-linux-arm-musleabihf": {
  "version": "4.60.4",
  "resolved": "****
"integrity": "sha512-
J3Yh9PzzF10vah2At+lHiGQdsYgArxBbXv/zHfSyaiFQEqvNv7DcW98pCrmdjCZBrqBiKrKKe2V+aaSGWuBe/w==",
"cpu": [
  "arm"
],
"libc": [
  "musl"
],
"license": "MIT",
"optional": true,
"os": [
  "linux"
]
},
"node_modules/@rollup/rollup-linux-arm64-gnu": {
  "version": "4.60.4",
  "resolved": "****
"integrity": "sha512-
BFDEZMYfUvLn37ONE1yMBojPxnMLTFsdyNoqncT0qFq1mAfLLL+ATMMJd8TeuVMiX84s1KbcxcZbXInmc02mRg==",
"cpu": [
  "arm64"
],
"libc": [
  "glibc"
],
},
```

```
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-arm64-musl": {
    "version": "4.60.4",
    "resolved": "****",
    "integrity": "sha512-
pc9EYOSlOgdQ2uPL1o9PF6/kLSgaUosia7g0uS8mB69IxJvclko1MECXysjs5ryez1/5zjYqx3+xYU0TU6R1A==",
    "cpu": [
      "arm64"
    ],
    "libc": [
      "musl"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-loong64-gnu": {
    "version": "4.60.4",
    "resolved": "****",
    "integrity": "sha512-
NxnomyxYerDh5n4iLrNa+sH+Z+U4BMEE46V2PgQ/hoB909i8gV1M5wPojWg9fk1jWp03IQn0s20K4wyZuFLEFQ==",
    "cpu": [
      "loong64"
    ],
    "libc": [
      "glibc"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-loong64-musl": {
    "version": "4.60.4",
    "resolved": "****",
    "integrity": "sha512-
nbJnQ8a3z1mtmrwImCYhc6BGpThAyYVRQxw9uKSKG4wR6aAYno9sVjJ0zaZcW9BPJX1GbrDPf+SvdWjgTuDmnw==",
    "cpu": [
      "loong64"
    ],
    "libc": [
      "musl"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-ppc64-gnu": {
    "version": "4.60.4",
    "resolved": "****",
    "integrity": "sha512-
2EU6acNrQLd8tYvo/LXW535wupT3m6fo7HKo6lr7ktQoItxTy0L1ZCR/GfGCuXl2vR+zmfI6eRXkSemaFv+iVg==",
    "cpu": [
      "ppc64"
    ],
    "libc": [
      "glibc"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
```

```

    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-ppc64-musl": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-WeBtoMuaMxiIr02IYP3xs6GMWkJP2C0EoT8beTLkUPmzV1i/Uc0SVw1d5r9KB0DtHKiLG5yFxsGRnBbK3wJ4A==",
  "cpu": [
    "ppc64"
  ],
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-riscv64-gnu": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-FJHFqpKUI3A10WrWkiFbBZ7yVbGT4q4B5o1qKFFojqpaYoh9LrQgqWcmmcxQzVSXYtyB5bzkXrYzLHTs21MYA==",
  "cpu": [
    "riscv64"
  ],
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-riscv64-musl": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-mcEL6CUT5IAUmQf1m9FYSmVqCJLpQ8r8eyftFUHG8i90hY7BkBXsUdnLH5D0f0wC0jcp9v/Q093zpmF1SptCCw==",
  "cpu": [
    "riscv64"
  ],
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-s390x-gnu": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-ynt3JxVd2w2buzoKDWIyiV1pJW93xlQic1THVLXiLz429oijRpSHivZAgp65KBu+cMcgf1eVVjdntLVPxgCuoQ==",
  "cpu": [
    "s390x"
  ],
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
}
},

```

```

"node_modules/@rollup/rollup-linux-x64-gnu": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-Boiz5+MsaR0EWDf+GGEwF8VMHGhLUoQMtIPj0gA5fv4osupqTVnJteQNKJwUcnUog2G55jYXH7KZFFiJe0TEzQ==",
  "cpu": [
    "x64"
  ],
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-x64-musl": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-qfSY27qIrFfI/Hom04KYFw3GKZSGU4lXus51wsb5EuySfFlWRwjkkWoE9emgRw/ukoT4Udsj4W/+xxG8VbPKg==",
  "cpu": [
    "x64"
  ],
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-openbsd-x64": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-VpTfOPHgVXEBeR8hZ200F3aSso+JDWqTwmTmzcQKted54IAdUVbxE+j/MVxUsKa8L20HJhv3vUezVPoquqWjA==",
  "cpu": [
    "x64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openbsd"
  ]
},
"node_modules/@rollup/rollup-openharmony-arm64": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-IP0sh5aRYuLv/nkU51X10Bf75BsF6+gZdx1X+QP5QM6LIJFHHqbHLG0uJn/hWthzo13UAc2umiUorqZy3axoZg==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "openharmony"
  ]
},
"node_modules/@rollup/rollup-win32-arm64-msvc": {
  "version": "4.60.4",
  "resolved": "****",
  "integrity": "sha512-4QzE9E810ohJ/HKzHhsqU+zcYyOjVOXLFMs1DdyMT6qXL/nioH7AVElmmEdUNHHS/oRkc++d5k6Vy85zFs0DEw==",
  "cpu": [
    "arm64"
  ],
  "license": "MIT",

```

```
    "optional": true,
    "os": [
      "win32"
    ]
  },
  "node_modules/@rollup/rollup-win32-ia32-msvc": {
    "version": "4.60.4",
    "resolved": "****",
    "integrity": "sha512-zTPgT1YuHHcd+Tmx7h8amLOFWFVeLV5N54oHow9SLj+GfoDy/huQ+UV396N/C7KpMDMiPspRktzM1/Or1usVEA==",
    "cpu": [
      "ia32"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ]
  },
  "node_modules/@rollup/rollup-win32-x64-gnu": {
    "version": "4.60.4",
    "resolved": "****",
    "integrity": "sha512-DRS4G7mi9LJxqEdeZIkKCaUIKCrLUUDCUaCsTPCi/rtqaC6D/jjwsLMQyiDU50Ka0JKpeXerBFBAxwArY52vBw==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ]
  },
  "node_modules/@rollup/rollup-win32-x64-msvc": {
    "version": "4.60.4",
    "resolved": "****",
    "integrity": "sha512-QVTUovf40zgTqLFVrKA1uXmVvU2QWFEWfAH8Wdc48IxLvrJMQVMBRjuQyUpzZCDkakImib9eVazbWLC6ksWtJw==",
    "cpu": [
      "x64"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ]
  },
  "node_modules/@types/estree": {
    "version": "1.0.8",
    "resolved": "****",
    "integrity": "sha512-dWHaZa2WqEXI/01E90jrocMTKJL2mSrEolh1Iomrv6U+JuNwaHXsXx9bLu5gG7BUWFIN0skIQJQ/L1rIex4X6w==",
    "license": "MIT"
  },
  "node_modules/@vitejs/plugin-vue": {
    "version": "5.2.4",
    "resolved": "****",
    "integrity": "sha512-7Yx/SXS0cQq5Hiiv3orevHUFn+pmMB4cgbEkDYgnkUWb0WfeQ/wa2yFv6D5ICiCQ0VpjA7vYDXrC7AG08yjDHA==",
    "license": "MIT",
    "engines": {
      "node": ">=18.0.0 || >=20.0.0"
    },
    "peerDependencies": {
      "vite": "^5.0.0 || ^6.0.0",
      "vue": "^3.2.25"
    }
  },
  "node_modules/@vue/compiler-core": {
    "version": "3.5.34",
    "resolved": "****"
```

```

    "integrity": "sha512-
s9cLyK5mLcvZ4Agva5QgRsQyLKvts9WbU9DB6NqiZkkGEdwmcEiylj5Jbwkp680drF/NNCV80LAJSe+yMLxaJw==",
    "license": "MIT",
    "dependencies": {
      "@babel/parser": "^7.29.3",
      "@vue/shared": "3.5.34",
      "entities": "^7.0.1",
      "estree-walker": "^2.0.2",
      "source-map-js": "^1.2.1"
    }
  },
  "node_modules/@vue/compiler-dom": {
    "version": "3.5.34",
    "resolved": "****",
    "integrity": "sha512-
EbF/T++k0e2MMZlJsBhzK8Sgwt0HcIP0hzn1CTB/lv6sQcyk+0Wf8YeilXzP3ro7MbbLcAfAJ6sEvjFWuNgUCw==",
    "license": "MIT",
    "dependencies": {
      "@vue/compiler-core": "3.5.34",
      "@vue/shared": "3.5.34"
    }
  },
  "node_modules/@vue/compiler-sfc": {
    "version": "3.5.34",
    "resolved": "****",
    "integrity": "sha512-
D/ihr6uZeIt6r+pVzf46RWt1fAsLFMBUP7k8G1VkiWexriED9GrX3echHd4Abbt17zjlfifJ8z7a3BxZ0PNjg==",
    "license": "MIT",
    "dependencies": {
      "@babel/parser": "^7.29.3",
      "@vue/compiler-core": "3.5.34",
      "@vue/compiler-dom": "3.5.34",
      "@vue/compiler-ssr": "3.5.34",
      "@vue/shared": "3.5.34",
      "estree-walker": "^2.0.2",
      "magic-string": "^0.30.21",
      "postcss": "^8.5.14",
      "source-map-js": "^1.2.1"
    }
  },
  "node_modules/@vue/compiler-ssr": {
    "version": "3.5.34",
    "resolved": "****",
    "integrity": "sha512-
cDtTHKibkThKGHH1SP+WdcquNRYQDFH6rRjQCqT9G2ltFAfor5pUftpab/z+aM5mW9HLLVQW7hfKKQe/1GBeQ==",
    "license": "MIT",
    "dependencies": {
      "@vue/compiler-dom": "3.5.34",
      "@vue/shared": "3.5.34"
    }
  },
  "node_modules/@vue/reactivity": {
    "version": "3.5.34",
    "resolved": "****",
    "integrity": "sha512-
y9XDjCEuBp+98k+UL5dbYkh57AHU4o6cxZed0PXw3bmrZZYLQsVHguGurq7hVrPCSrQtrnz1f9dssyFr+dMXfQ==",
    "license": "MIT",
    "dependencies": {
      "@vue/shared": "3.5.34"
    }
  },
  "node_modules/@vue/runtime-core": {
    "version": "3.5.34",
    "resolved": "****",
    "integrity": "sha512-
mKeBYvu8tcMSLhypAHBmrUFWXTCTF/23Z4jiCoYK3UtWepkLiViNLuR90V9X0yD62mUxs9p1jsrpK3CCGIzw==",
    "license": "MIT",
    "dependencies": {
      "@vue/reactivity": "3.5.34",
      "@vue/shared": "3.5.34"
    }
  }

```

```
    },
    "node_modules/@vue/runtime-dom": {
      "version": "3.5.34",
      "resolved": "****",
      "integrity": "sha512-e8kZzERmCwUnBRVsgSQLAfrfU2rGoyOFFKPBXSLfEjc/03KfA7QP0t1/2ZylrbchjmIKB4dPTd07A6WPr0eOrg==",
      "license": "MIT",
      "dependencies": {
        "@vue/reactivity": "3.5.34",
        "@vue/runtime-core": "3.5.34",
        "@vue/shared": "3.5.34",
        "csstype": "^3.2.3"
      }
    },
    "node_modules/@vue/server-renderer": {
      "version": "3.5.34",
      "resolved": "****",
      "integrity": "sha512-nHxmJotTrKsmrkbILRhKc9gY1G3moZbJTqCzDd7D00zG5KH9oeJ0Unqrf5f9v0pW//jES05ZkJcNtfE8Jj0Iew==",
      "license": "MIT",
      "dependencies": {
        "@vue/compiler-ssr": "3.5.34",
        "@vue/shared": "3.5.34"
      },
      "peerDependencies": {
        "vue": "3.5.34"
      }
    },
    "node_modules/@vue/shared": {
      "version": "3.5.34",
      "resolved": "****",
      "integrity": "sha512-24uqU40IiX29ryC3MeWid/Xf2fa2EFRUvLb77nRhk+UrTVrh/XiGtFAFmJBAtBRbjwNdsPRP+jj/0L27Eg1NDA==",
      "license": "MIT"
    },
    "node_modules/csstype": {
      "version": "3.2.3",
      "resolved": "****",
      "integrity": "sha512-z1HGKcYy2xA8AGQfwrn0PAY+PB7X/GSj3UVJW9qKyn43xWa+gL5nXmU4qqLMRzWVLF8KusUX8T/0kCi0YpAIQ==",
      "license": "MIT"
    },
    "node_modules/entities": {
      "version": "7.0.1",
      "resolved": "****",
      "integrity": "sha512-TwrgLOFUTTh994YUyl1yT4uyavY5nNB5muff+RtWaqNVCAK408b5ZnnbNAUEWLTCpum9w6arT70i1XdQ4UeOPA==",
      "license": "BSD-2-Clause",
      "engines": {
        "node": ">=0.12"
      },
      "funding": {
        "url": "****"
      }
    },
    "node_modules/esbuild": {
      "version": "0.25.12",
      "resolved": "****",
      "integrity": "sha512-bbPBYrtZbkt60s6FiTLCTFxvq4tt3JKallvRwshA3fdVztsLAatFaZobhkBC8/BrPetoa0oksYoKXoG4ryJg==",
      "hasInstallScript": true,
      "license": "MIT",
      "bin": {
        "esbuild": "bin/esbuild"
      },
      "engines": {
        "node": ">=18"
      },
      "optionalDependencies": {
        "@esbuild/aix-ppc64": "0.25.12",
        "@esbuild/android-arm": "0.25.12",

```

```

"@esbuild/android-arm64": "0.25.12",
"@esbuild/android-x64": "0.25.12",
"@esbuild/darwin-arm64": "0.25.12",
"@esbuild/darwin-x64": "0.25.12",
"@esbuild/freebsd-arm64": "0.25.12",
"@esbuild/freebsd-x64": "0.25.12",
"@esbuild/linux-arm": "0.25.12",
"@esbuild/linux-arm64": "0.25.12",
"@esbuild/linux-ia32": "0.25.12",
"@esbuild/linux-loong64": "0.25.12",
"@esbuild/linux-mips64el": "0.25.12",
"@esbuild/linux-ppc64": "0.25.12",
"@esbuild/linux-riscv64": "0.25.12",
"@esbuild/linux-s390x": "0.25.12",
"@esbuild/linux-x64": "0.25.12",
"@esbuild/netbsd-arm64": "0.25.12",
"@esbuild/netbsd-x64": "0.25.12",
"@esbuild/openbsd-arm64": "0.25.12",
"@esbuild/openbsd-x64": "0.25.12",
"@esbuild/openharmony-arm64": "0.25.12",
"@esbuild/sunos-x64": "0.25.12",
"@esbuild/win32-arm64": "0.25.12",
"@esbuild/win32-ia32": "0.25.12",
"@esbuild/win32-x64": "0.25.12"
},
},
"node_modules/estree-walker": {
  "version": "2.0.2",
  "resolved": "****",
  "integrity": "sha512-Rfkk/Mp/DL7JVje3u18FxFuJQLTNR2q6QfMSMB7AvCBx91NGj/ba3kCfza0f6dVDbw7YLRf/nDrn7pQrCCyQ/w==",
  "license": "MIT"
},
"node_modules/fdir": {
  "version": "6.5.0",
  "resolved": "****",
  "integrity": "sha512-tIbYtZbuc0s0BRGqPJkshJUYdL+SDH7dVM8gjy+ERp3WAUjLEFJE+O2kanyHtwjWOnwrKYBiwAmM0p4kLJAnXg==",
  "license": "MIT",
  "engines": {
    "node": ">=12.0.0"
  },
  "peerDependencies": {
    "picomatch": "^3 || ^4"
  },
  "peerDependenciesMeta": {
    "picomatch": {
      "optional": true
    }
  }
},
"node_modules/fsevents": {
  "version": "2.3.3",
  "resolved": "****",
  "integrity": "sha512-5xuDfX+fl7faATnagmWPpbFtwh/R77WmMMqqHGS65C3vvB0YHrgF+B1YmZ3441tMj5n63k0212XNoJwzLhffQw==",
  "hasInstallScript": true,
  "license": "MIT",
  "optional": true,
  "os": [
    "darwin"
  ],
  "engines": {
    "node": "^8.16.0 || ^10.6.0 || >=11.0.0"
  }
},
"node_modules/magic-string": {
  "version": "0.30.21",
  "resolved": "****",
  "integrity": "sha512-vd2F4YUyEXKGCILHoq+TEyCjxueSeHnFxyjNp80ygoXV4vUhnDer/LvvLqM/arB5bXQN5K2/3oinyCRyx8T2CQ==",

```

```

    "license": "MIT",
    "dependencies": {
      "@jridgewell/sourcemap-codec": "^1.5.5"
    }
  },
  "node_modules/nanoid": {
    "version": "3.3.12",
    "resolved": "****",
    "integrity": "sha512-
ZB9RH/39qpq5Vu6Y+NmUaFhQR6pp+M2Xt76XBnEwDaGcVAqhLvxl3B2bKS5D3NH3QR76v3aSrKaF/Kiy7LEtQ==",
    "funding": [
      {
        "type": "github",
        "url": "****"
      }
    ],
    "license": "MIT",
    "bin": {
      "nanoid": "bin/nanoid.cjs"
    },
    "engines": {
      "node": "^10 || ^12 || ^13.7 || ^14 || >=15.0.1"
    }
  },
  "node_modules/picocolors": {
    "version": "1.1.1",
    "resolved": "****",
    "integrity": "sha512-
xceH2snhtb5M9LiQDsmEw56le376mTZkEX/jEb/RxNFyegNuL7eNsLCXP9FDj/Lcu0X8KEyMceP2ntpaHrDEVA==",
    "license": "ISC"
  },
  "node_modules/picomatch": {
    "version": "4.0.4",
    "resolved": "****",
    "integrity": "sha512-
QP88BAKvMam/3NxH6vj2o21R6MjxZUAd6nLwAS/pnGvN9IVLocLHxGYIzFhg6fUQ+5th6P4dv4ew9jX3DSIj7A==",
    "license": "MIT",
    "engines": {
      "node": ">=12"
    },
    "funding": {
      "url": "****"
    }
  },
  "node_modules/postcss": {
    "version": "8.5.15",
    "resolved": "****",
    "integrity": "sha512-
FfR8sjd4em2T6fb3I2MwAJU7HWVMr9zba+enmQeeWfCbm+UOC/0X4DS8XtpUTMwWMGbjKYP7xj fNekzyGmB3A==",
    "funding": [
      {
        "type": "opencollective",
        "url": "****"
      },
      {
        "type": "tidelift",
        "url": "****"
      },
      {
        "type": "github",
        "url": "****"
      }
    ],
    "license": "MIT",
    "dependencies": {
      "nanoid": "^3.3.12",
      "picocolors": "^1.1.1",
      "source-map-js": "^1.2.1"
    },
    "engines": {
      "node": "^10 || ^12 || >=14"
    }
  }

```

```

    },
    "node_modules/rollup": {
      "version": "4.60.4",
      "resolved": "****",
      "integrity": "sha512-
WHeFSbZYsPu3+bLoNRUuA0+wavNLocOPf3wSHTP7hcFKVnJewsYLCDBr3mTS14FCizf9ccIXA8sGL8zKeQN3g==",
      "license": "MIT",
      "dependencies": {
        "@types/estree": "1.0.8"
      },
      "bin": {
        "rollup": "dist/bin/rollup"
      },
      "engines": {
        "node": ">=18.0.0",
        "npm": ">=8.0.0"
      },
      "optionalDependencies": {
        "@rollup/rollup-android-arm-eabi": "4.60.4",
        "@rollup/rollup-android-arm64": "4.60.4",
        "@rollup/rollup-darwin-arm64": "4.60.4",
        "@rollup/rollup-darwin-x64": "4.60.4",
        "@rollup/rollup-freebsd-arm64": "4.60.4",
        "@rollup/rollup-freebsd-x64": "4.60.4",
        "@rollup/rollup-linux-arm-gnueabihf": "4.60.4",
        "@rollup/rollup-linux-arm-musleabihf": "4.60.4",
        "@rollup/rollup-linux-arm64-gnu": "4.60.4",
        "@rollup/rollup-linux-arm64-musl": "4.60.4",
        "@rollup/rollup-linux-loong64-gnu": "4.60.4",
        "@rollup/rollup-linux-loong64-musl": "4.60.4",
        "@rollup/rollup-linux-ppc64-gnu": "4.60.4",
        "@rollup/rollup-linux-ppc64-musl": "4.60.4",
        "@rollup/rollup-linux-riscv64-gnu": "4.60.4",
        "@rollup/rollup-linux-riscv64-musl": "4.60.4",
        "@rollup/rollup-linux-s390x-gnu": "4.60.4",
        "@rollup/rollup-linux-x64-gnu": "4.60.4",
        "@rollup/rollup-linux-x64-musl": "4.60.4",
        "@rollup/rollup-openbsd-x64": "4.60.4",
        "@rollup/rollup-openharmony-arm64": "4.60.4",
        "@rollup/rollup-win32-arm64-msvc": "4.60.4",
        "@rollup/rollup-win32-ia32-msvc": "4.60.4",
        "@rollup/rollup-win32-x64-gnu": "4.60.4",
        "@rollup/rollup-win32-x64-msvc": "4.60.4",
        "fsevents": "~2.3.2"
      }
    },
    "node_modules/source-map-js": {
      "version": "1.2.1",
      "resolved": "****",
      "integrity": "sha512-
UXWMKhLowVKb728IUtQPXxfU+usdybtUrK/8uGE8CQMvrh0pwwzDBwj0QhSL7MQc7vIsISBG8VQ8+IDQxpfQA==",
      "license": "BSD-3-Clause",
      "engines": {
        "node": ">=0.10.0"
      }
    },
    "node_modules/tingylobby": {
      "version": "0.2.16",
      "resolved": "****",
      "integrity": "sha512-
pn99VhoACYR8nFHxqix+uvsbXineAasWm5ojXoN8xEwK5Kd3/TrhNn1wByuD52UxWRLy8pu+kRMniEi6Eq9Zg==",
      "license": "MIT",
      "dependencies": {
        "fdir": "^6.5.0",
        "picomatch": "^4.0.4"
      },
      "engines": {
        "node": ">=12.0.0"
      },
      "funding": {

```

```
    "url": "***"
  },
  "node_modules/vite": {
    "version": "6.4.2",
    "resolved": "***",
    "integrity": "sha512-2N/55r4JDJ4gdrCvGgINMy+HH3iRpNIz8K6SFwVsA+JbQScLiC+cLmAxBgwiSPgcG9U15QmvqCGWzMbqda5zGQ==",
    "license": "MIT",
    "dependencies": {
      "esbuild": "^0.25.0",
      "fdir": "^6.4.4",
      "picomatch": "^4.0.2",
      "postcss": "^8.5.3",
      "rollup": "^4.34.9",
      "tinyglobby": "^0.2.13"
    },
    "bin": {
      "vite": "bin/vite.js"
    },
    "engines": {
      "node": "^18.0.0 || ^20.0.0 || >=22.0.0"
    },
    "funding": {
      "url": "***"
    },
    "optionalDependencies": {
      "fsevents": "~2.3.3"
    },
    "peerDependencies": {
      "@types/node": "^18.0.0 || ^20.0.0 || >=22.0.0",
      "jiti": ">=1.21.0",
      "less": "*",
      "lightningcss": "^1.21.0",
      "sass": "*",
      "sass-embedded": "*",
      "stylus": "*",
      "sugarss": "*",
      "terser": "^5.16.0",
      "tsx": "^4.8.1",
      "yaml": "^2.4.2"
    },
    "peerDependenciesMeta": {
      "@types/node": {
        "optional": true
      },
      "jiti": {
        "optional": true
      },
      "less": {
        "optional": true
      },
      "lightningcss": {
        "optional": true
      },
      "sass": {
        "optional": true
      },
      "sass-embedded": {
        "optional": true
      },
      "stylus": {
        "optional": true
      },
      "sugarss": {
        "optional": true
      },
      "terser": {
        "optional": true
      },
      "tsx": {
```

```

      "optional": true
    },
    "yaml": {
      "optional": true
    }
  },
  "node_modules/vue": {
    "version": "3.5.34",
    "resolved": "****",
    "integrity": "sha512-WdLBG9gm020gJIG9axd5Hpx0TFLdzVgfg62evFFu8Rur50/IoGc5cMjnjh3tPL6GnRGsYvUhbSKVPYVcxRKpMCA==",
    "license": "MIT",
    "dependencies": {
      "@vue/compiler-dom": "3.5.34",
      "@vue/compiler-sfc": "3.5.34",
      "@vue/runtime-dom": "3.5.34",
      "@vue/server-renderer": "3.5.34",
      "@vue/shared": "3.5.34"
    },
    "peerDependencies": {
      "typescript": "*"
    },
    "peerDependenciesMeta": {
      "typescript": {
        "optional": true
      }
    }
  }
}

```

frontend/package.json

```

{
  "name": "railway-power-operation-video-ai-evaluator",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@vitejs/plugin-vue": "^5.2.1",
    "vite": "^6.0.3",
    "vue": "^3.5.13"
  },
  "devDependencies": {}
}

```

frontend/src/App.vue

```

<script setup>
import { computed, onMounted, ref } from 'vue'
import { api } from './services/api'

const loading = ref(false)
const message = ref('')
const stats = ref({ total_jobs: 0, completed_jobs: 0, average_score: 0, excellent_count: 0, risk_count: 0 })
const tasks = ref([])
const jobs = ref([])
const steps = ref([])
const rules = ref([])
const knowledge = ref([])
const selectedJobId = ref(null)
const detail = ref(null)

```

```

const fileInput = ref(null)

const taskForm = ref({
  title: '接触网停电验电接地实训评价',
  course: '铁道供电安全实训',
  class_name: '供电 2401 班',
  teacher: '张老师',
  description: '围绕防护用品、工作票确认、验电、挂接地线、开关操作和复核状态进行视频智能评价。'
})

const uploadForm = ref({
  task_id: '',
  student_name: '李明',
  student_no: '20260001'
})

const selectedTask = computed(() => tasks.value.find((task) => task.id === Number(uploadForm.value.task_id)))
const completedJobs = computed(() => jobs.value.filter((job) => job.status === 'completed'))

function formatTime(seconds) {
  if (seconds === null || seconds === undefined) return '-'
  const minute = Math.floor(seconds / 60)
  const second = Math.floor(seconds % 60)
  return `${String(minute).padStart(2, '0')}:${String(second).padStart(2, '0')}`
}

async function refreshAll() {
  const [nextStats, nextTasks, nextJobs, nextSteps, nextRules, nextKnowledge] = await Promise.all([
    api.stats(),
    api.listTasks(),
    api.listJobs(),
    api.steps(),
    api.rules(),
    api.knowledge()
  ])
  stats.value = nextStats
  tasks.value = nextTasks
  jobs.value = nextJobs
  steps.value = nextSteps
  rules.value = nextRules
  knowledge.value = nextKnowledge
  if (!uploadForm.value.task_id && nextTasks.length) {
    uploadForm.value.task_id = String(nextTasks[0].id)
  }
  if (!selectedJobId.value && nextJobs.length) {
    await selectJob(nextJobs[0].id)
  } else if (selectedJobId.value) {
    await selectJob(selectedJobId.value)
  }
}

async function createTask() {
  loading.value = true
  message.value = ''
  try {
    const task = await api.createTask(taskForm.value)
    uploadForm.value.task_id = String(task.id)
    message.value = '评价任务已创建'
    await refreshAll()
  } catch (error) {
    message.value = error.message
  } finally {
    loading.value = false
  }
}

async function uploadVideo() {
  const file = fileInput.value?.files?.[0]
  if (!file) {
    message.value = '请先选择视频文件'
    return
  }
}

```

```

    }
    if (!uploadForm.value.task_id) {
      message.value = '请先创建或选择评价任务'
      return
    }
    loading.value = true
    message.value = ''
    try {
      const formData = new FormData()
      formData.append('task_id', uploadForm.value.task_id)
      formData.append('student_name', uploadForm.value.student_name)
      formData.append('student_no', uploadForm.value.student_no)
      formData.append('file', file)
      const result = await api.uploadSubmission(formData)
      selectedJobId.value = result.job.id
      message.value = '视频已上传, 分析任务已创建'
      await new Promise((resolve) => setTimeout(resolve, 600))
      await refreshAll()
    } catch (error) {
      message.value = error.message
    } finally {
      loading.value = false
    }
  }
}

async function selectJob(jobId) {
  selectedJobId.value = jobId
  detail.value = await api.getDetail(jobId)
}

async function rerunJob() {
  if (!selectedJobId.value) return
  loading.value = true
  try {
    await api.rerun(selectedJobId.value)
    message.value = '已重新分析'
    await refreshAll()
  } catch (error) {
    message.value = error.message
  } finally {
    loading.value = false
  }
}

onMounted(async () => {
  loading.value = true
  try {
    await refreshAll()
  } catch (error) {
    message.value = `后端连接失败: ${error.message}`
  } finally {
    loading.value = false
  }
})
</script>

<template>
  <main class="app-shell">
    <header class="topbar">
      <div>
        <p class="eyebrow">Railway Power Operation AI Evaluator</p>
        <h1>铁道供电作业视频智能评价平台</h1>
      </div>
      <button class="ghost-button" :disabled="loading" @click="refreshAll">刷新</button>
    </header>

    <section class="metrics">
      <div class="metric">
        <span>分析任务</span>
        <strong>{{ stats.total_jobs }}</strong>
      </div>

```

```
<div class="metric">
  <span>已完成</span>
  <strong>{{ stats.completed_jobs }}</strong>
</div>
<div class="metric">
  <span>平均分</span>
  <strong>{{ stats.average_score }}</strong>
</div>
<div class="metric">
  <span>风险作业</span>
  <strong>{{ stats.risk_count }}</strong>
</div>
</section>
```

```
<p v-if="message" class="message">{{ message }}</p>
```

```
<section class="workspace">
  <aside class="panel controls">
    <h2>基本操作</h2>
    <div class="form-grid">
      <label>
        <span>任务名称</span>
        <input v-model="taskForm.title" />
      </label>
      <label>
        <span>课程</span>
        <input v-model="taskForm.course" />
      </label>
      <label>
        <span>班级</span>
        <input v-model="taskForm.class_name" />
      </label>
      <label>
        <span>教师</span>
        <input v-model="taskForm.teacher" />
      </label>
      <label class="full">
        <span>任务说明</span>
        <textarea v-model="taskForm.description" rows="3"></textarea>
      </label>
    </div>
    <button class="primary-button" :disabled="loading" @click="createTask">创建评价任务</button>
  </div>
  <div class="divider"></div>
```

```
<label>
  <span>选择任务</span>
  <select v-model="uploadForm.task_id">
    <option v-for="task in tasks" :key="task.id" :value="String(task.id)">
      {{ task.title }}
    </option>
  </select>
</label>
```

```
<div class="form-grid compact">
  <label>
    <span>学生姓名</span>
    <input v-model="uploadForm.student_name" />
  </label>
  <label>
    <span>学号</span>
    <input v-model="uploadForm.student_no" />
  </label>
</div>
```

```
<label>
  <span>上传视频</span>
  <input ref="fileInput" type="file" accept="video/*,.txt,.mp4,.mov,.avi" />
</label>
<button class="primary-button" :disabled="loading" @click="uploadVideo">上传并分析</button>
```

```
<p v-if="selectedTask" class="hint">当前任务: {{ selectedTask.class_name }} / {{ selectedTask.teacher }}
```

```
</p>
```

```
</aside>
```

```

<section class="panel result-panel">
  <div class="section-header">
    <h2>评分结果</h2>
    <button class="ghost-button" :disabled="!selectedJobId || loading" @click="rerunJob">重新分析</button>
  </div>

  <div class="job-list">
    <button
      v-for="job in jobs"
      :key="job.id"
      class="job-item"
      :class="{ active: job.id === selectedJobId }"
      @click="selectJob(job.id)"
    >
      <span>#{{ job.id }} {{ job.status }}</span>
      <strong>{{ job.score }} 分</strong>
    </button>
  </div>

  <div v-if="detail" class="detail-grid">
    <div class="video-pane">
      <video
        v-if="detail.submission"
        controls
        :src="api.videoUrl(detail.submission.id)"
      ></video>
      <div class="report-card">
        <span class="score">{{ detail.job.score }}</span>
        <div>
          <h3>{{ detail.report?.conclusion || detail.job.summary || '等待分析完成' }}</h3>
          <p>{{ detail.report?.strengths }}</p>
        </div>
      </div>
    </div>

    <div class="evidence-pane">
      <h3>标准流程识别</h3>
      <ol class="timeline">
        <li v-for="step in detail.steps" :key="step.id">
          <time>{{ formatTime(step.start_sec) }}</time>
          <div>
            <strong>{{ step.step_name }}</strong>
            <p>{{ step.evidence }}</p>
          </div>
        </li>
      </ol>

      <h3>扣分项</h3>
      <div v-if="detail.violations.length" class="violations">
        <article v-for="item in detail.violations" :key="item.id" class="violation">
          <div>
            <strong>{{ item.title }}</strong>
            <span>{{ item.severity }} / 扣 {{ item.deduction }} 分</span>
          </div>
          <p>{{ item.reason }}</p>
          <small>{{ item.suggestion }}</small>
        </article>
      </div>
      <p v-else class="empty">暂无硬性规则扣分项</p>
    </div>

    <div v-else class="empty-state">
      创建任务并上传视频后，这里会显示 AI 证据、扣分项和评价报告。
    </div>
  </section>
</section>

<section class="reference-grid">
  <div class="panel">

```

```

<h2>评分规则</h2>
<ul class="plain-list">
  <li v-for="rule in rules" :key="rule.id">
    <strong>{{ rule.title }}</strong>
    <span>扣 {{ rule.deduction }} 分</span>
  </li>
</ul>
</div>
<div class="panel">
  <h2>标准步骤</h2>
  <ul class="plain-list">
    <li v-for="step in steps" :key="step.id">
      <strong>{{ step.order_index }}. {{ step.name }}</strong>
      <span>{{ step.required ? '必需' : '可选' }}</span>
    </li>
  </ul>
</div>
<div class="panel">
  <h2>知识库</h2>
  <ul class="plain-list docs">
    <li v-for="doc in knowledge" :key="doc.id">
      <strong>{{ doc.title }}</strong>
      <span>{{ doc.category }}</span>
    </li>
  </ul>
</div>
</section>
</main>
</template>

```

frontend/src/main.js

```

import { createApp } from 'vue'
import App from './App.vue'
import './styles.css'

createApp(App).mount('#app')

```

frontend/src/services/api.js

```

const API_BASE = import.meta.env.VITE_API_BASE || '***'

async function request(path, options = {}) {
  const response = await fetch(`${API_BASE}${path}`, options)
  if (!response.ok) {
    const detail = await response.text()
    throw new Error(detail || `HTTP ${response.status}`)
  }
  return response.json()
}

export const api = {
  base: API_BASE,
  health: () => request('/api/health'),
  stats: () => request('/api/analysis/stats/overview'),
  listTasks: () => request('/api/tasks'),
  createTask: (payload) =>
    request('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    }),
  listSubmissions: (taskId) => request(`/api/submissions${taskId ? `?task_id=${taskId}` : ''}`),
  listJobs: () => request('/api/analysis/jobs'),
  getDetail: (jobId) => request(`/api/analysis/jobs/${jobId}/detail`),
  rerun: (jobId) => request(`/api/analysis/jobs/${jobId}/rerun`, { method: 'POST' }),
  steps: () => request('/api/catalog/steps'),
  rules: () => request('/api/catalog/rules'),
}

```

```
knowledge: () => request('/api/catalog/knowledge'),
videoUrl: (submissionId) => `${API_BASE}/api/submissions/${submissionId}/video`,
uploadSubmission: (formData) =>
  request('/api/submissions', {
    method: 'POST',
    body: formData
  })
}
```

frontend/src/styles.css

```
:root {
  font-family: "Inter", "PingFang SC", "Microsoft YaHei", Arial, sans-serif;
  color: #17202a;
  background: #f5f7fa;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
}

button,
input,
select,
textarea {
  font: inherit;
}

.app-shell {
  min-height: 100vh;
  padding: 24px;
}

.topbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 20px;
}

.eyebrow {
  margin: 0 0 6px;
  color: #607084;
  font-size: 13px;
}

h1,
h2,
h3,
p {
  margin-top: 0;
}

h1 {
  margin-bottom: 0;
  font-size: clamp(26px, 4vw, 40px);
  letter-spacing: 0;
}

h2 {
  font-size: 18px;
  margin-bottom: 16px;
}
```

```
}

h3 {
  font-size: 16px;
  margin-bottom: 8px;
}

.metrics {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 12px;
  margin-bottom: 16px;
}

.metric,
.panel {
  background: #ffffff;
  border: 1px solid #dde4ed;
  border-radius: 8px;
}

.metric {
  padding: 16px;
}

.metric span {
  display: block;
  color: #607084;
  margin-bottom: 8px;
}

.metric strong {
  font-size: 30px;
}

.message {
  padding: 12px 14px;
  border-radius: 8px;
  background: #fff7e6;
  border: 1px solid #f0c36d;
  color: #6f4d00;
}

.workspace {
  display: grid;
  grid-template-columns: 360px minmax(0, 1fr);
  gap: 16px;
  align-items: start;
}

.panel {
  padding: 18px;
}

.controls {
  position: sticky;
  top: 16px;
}

label {
  display: grid;
  gap: 6px;
  margin-bottom: 12px;
  color: #44546a;
}

input,
select,
textarea {
  width: 100%;
  border: 1px solid #cbd5e1;
}
```

```
border-radius: 6px;
padding: 10px 12px;
background: #ffffff;
color: #17202a;
}

textarea {
  resize: vertical;
}

.form-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
}

.form-grid.compact {
  gap: 10px;
}

.full {
  grid-column: 1 / -1;
}

.primary-button,
.ghost-button {
  min-height: 40px;
  border-radius: 6px;
  border: 1px solid transparent;
  padding: 0 14px;
  cursor: pointer;
}

.primary-button {
  width: 100%;
  background: #176b87;
  color: #ffffff;
}

.ghost-button {
  background: #ffffff;
  border-color: #cbd5e1;
  color: #243447;
}

button:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.divider {
  height: 1px;
  background: #e5eaf0;
  margin: 18px 0;
}

.hint,
.empty,
.empty-state {
  color: #607084;
}

.result-panel {
  min-height: 620px;
}

.section-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}
```

```
}

.job-list {
  display: flex;
  gap: 8px;
  overflow-x: auto;
  padding-bottom: 10px;
  margin-bottom: 16px;
}

.job-item {
  display: flex;
  align-items: center;
  gap: 10px;
  white-space: nowrap;
  min-height: 38px;
  padding: 0 12px;
  border-radius: 6px;
  border: 1px solid #d5dde7;
  background: #ffffff;
}

.job-item.active {
  border-color: #176b87;
  background: #e8f6fa;
}

.detail-grid {
  display: grid;
  grid-template-columns: minmax(280px, 44%) minmax(0, 1fr);
  gap: 18px;
}

.video-pane video {
  width: 100%;
  aspect-ratio: 16 / 9;
  background: #111827;
  border-radius: 8px;
  border: 1px solid #d5dde7;
}

.report-card {
  display: flex;
  gap: 14px;
  margin-top: 14px;
  padding: 14px;
  border-radius: 8px;
  border: 1px solid #d5dde7;
  background: #f8fafc;
}

.score {
  display: grid;
  place-items: center;
  width: 68px;
  height: 68px;
  flex: 0 0 auto;
  border-radius: 50%;
  background: #176b87;
  color: #ffffff;
  font-size: 24px;
  font-weight: 700;
}

.timeline {
  list-style: none;
  padding: 0;
  margin: 0 0 18px;
}

.timeline li {
```

```
display: grid;
grid-template-columns: 58px minmax(0, 1fr);
gap: 12px;
padding: 10px 0;
border-bottom: 1px solid #edf1f5;
}

.timeline time {
  color: #176b87;
  font-weight: 700;
}

.timeline p,
.violation p,
.report-card p {
  margin-bottom: 0;
  color: #607084;
  line-height: 1.55;
}

.violations {
  display: grid;
  gap: 10px;
}

.violation {
  border: 1px solid #f0c1bd;
  background: #fff7f6;
  border-radius: 8px;
  padding: 12px;
}

.violation div {
  display: flex;
  justify-content: space-between;
  gap: 12px;
}

.violation span,
.violation small {
  color: #8a3b35;
}

.reference-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 16px;
  margin-top: 16px;
}

.plain-list {
  list-style: none;
  padding: 0;
  margin: 0;
  display: grid;
  gap: 8px;
}

.plain-list li {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 10px;
  border-radius: 6px;
  background: #f8fafc;
}

.plain-list span {
  color: #607084;
  white-space: nowrap;
```

```
}

.docs li {
  align-items: flex-start;
}

@media (max-width: 1120px) {
  .workspace,
  .detail-grid,
  .reference-grid {
    grid-template-columns: 1fr;
  }

  .controls {
    position: static;
  }
}

@media (max-width: 720px) {
  .app-shell {
    padding: 14px;
  }

  .topbar,
  .section-header,
  .violation div {
    align-items: stretch;
    flex-direction: column;
  }

  .metrics,
  .form-grid {
    grid-template-columns: 1fr;
  }
}
```

frontend/vite.config.js

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  server: {
    host: '***',
    port: 5173
  }
})
```